

FaceGuard

Access Control System with Face Recognition

Team 17

Project Context

Problem

University lab uses manual key-based access. It is slow, the card is easy to lose.

There is no automatic way to log who enters or to block unauthorized visitors.

Solution

FaceGuard runs on a Raspberry Pi 5 and uses face recognition to grant or deny access automatically.

Every attempt is logged. No keys are needed.

Stakeholders

Customer

Artyom Tuzov
University lab worker
Defines requirements and priorities

End Users

Lab employees
Use the system daily
Need fast, reliable access

Development Team

6 students
Design, build, and maintain
the system



Demo

Team

Sofia Sokolova

Scrum Master / Backend

- Reports and documentation
- Backend
- UAT
- Database
- Scrum facilitation
- Acceptance criteria and Definition of Done

Varvara Orekhova

Product Owner / Backend

- Reports and documentation
- Backend
- Database
- Product backlog prioritization
- Customer communication and feedback
- Face recognition model integration

Maksim Beketov

Developer / Backend, CI

- Prototype design
- Face recognition pipeline integration
- Reports and documentation
- CI pipeline configuration
- QRT and performance testing in CI

Maksim Barannikov

Developer / Backend, Testing

- Repository workflow configuration
- Face recognition pipeline integration
- Unit and integration testing
- QRT implementation
- Reports and documentation
- Hardware setup

Alexander Bondarenko

Developer / Embedded

- Reports and documentation
- Hardware setup
- Camera module integration
- Quality requirements definition and QRT implementation

Pavel Dudinov

Developer / Technical writer

- Interview scripts and transcripts
- User story migration and formatting
- Customer feedback response documentation

Requirements

Functional (MoSCoW)

Must: face registration, accurate auto recognition, access grant/deny, database up to 40 people

Should: view employee list, recognition with accessories

Could: temporary access, access logs, liveness detection, failure notifications

Won't: recognize a person in a crowd

Quality Requirements

QR-1 Time Behaviour

Recognition result ≤ 3 seconds for up to 20 registered users

QR-2 Security

System rejects at least 9 out of 10 2D photo attacks, all unregistered users during testing, grants access to all registered users during testing

QR-3 Modularity

Recognition model can be swapped without changing API, UI, or business logic

Roadmap

Sprint 1 (Done)

- Face recognition
- Camera integration via OpenCV
- Employee registration (permanent access)
- Employee removal
- Admin panel (authentication, employee list, access logs)
- PostgreSQL database
- Docker containerization
- Local setup instructions

Sprint 2 (Now)

- Lightweight face recognition model
- Automatic recognition (no button press)
- Video-based employee registration
- Raspberry Pi 5 setup and camera integration
- Quality Requirements and QRT implementation
- CI pipeline with automated tests and coverage
- Liveness detection

Sprints 3, 4, 5 (Next)

- LED indicators for access grant/deny
- Recognition with accessories (glasses, masks)
- Recognition accuracy improvements
- Smart lock integration
- Failure notifications
- Final polish and production readiness

Client Collaboration

Already in use

Tested and approved by the client (UAT session):

- Face registration
- Face recognition
- Admin panel
- Access logs

Currently in testing phase, not yet actively used in production.

Useful feedback

- Wants video-based registration
- Wants to choose start and expiration dates for temporary access instead of the number of days

Current challenges

- Recognition speed is degraded and the site freezes due to the performance limitations of the Raspberry Pi 5

Project Link

GitHub Repository & Deployed Application

<https://github.com/Innopolis-Robotics-Society/FaceGuardV3.git>

